

S3

Sémantique SQL – chapitre 1

GROUP BY : changer de granularité

Après un regroupement, une ligne du résultat ne représente plus un film mais un paquet de films.

TU ES ICI SI...

- ▶ « PostgreSQL me dit qu'une colonne *must appear in the GROUP BY clause*. »
- ▶ « Je ne sais jamais quoi mettre dans le **GROUP BY**. »
- ▶ « J'ai regroupé, et maintenant je ne peux plus afficher le titre du film. »
- ▶ « Combien de lignes ma requête va-t-elle renvoyer ? »

GROUP BY range les lignes en **paquets**, puis réduit chaque paquet à **une seule ligne**. Le résultat change donc de nature : avant, une ligne était un film ; après, une ligne est un *ensemble* de films. Tout ce que vous affichez doit être une propriété du **paquet**, pas d'un film particulier.

Le mécanisme en deux temps

1. On range en paquets...

title	length	rating
Academy Dinosaur	86	PG
Bucket Brotherhood	133	PG
Frost Head	78	PG
Escape Metropolis	141	R
Hunting Musketeers	65	R
Island Empire	NULL	PG-13

Trois valeurs distinctes de rating → trois paquets.

2. ... puis chaque paquet devient une ligne

```
SELECT rating, COUNT(*) AS nb, AVG(length) AS moy
FROM film
GROUP BY rating;
```

rating	nb	moy
PG	3	99.0
R	2	103.0
PG-13	1	NULL

Le nombre de lignes du résultat est le nombre de **valeurs distinctes** de la colonne de regroupement. Ici : 3.

La règle du SELECT, et pourquoi elle existe

✗ CE QUE VOUS AVEZ SANS DOUTE ÉCRIT

```
SELECT rating, title, COUNT(*)
FROM film
GROUP BY rating;
```

Le paquet PG contient *trois* titres. Lequel PostgreSQL devrait-il afficher ? La question n'a pas de réponse, donc la requête est refusée.

✓ CE QU'IL FAUT ÉCRIRE

```
SELECT rating, COUNT(*), MIN(title), MAX(length)
FROM film
GROUP BY rating;
```

Chaque expression donne **une** valeur par paquet : le nombre de films, le premier titre par ordre alphabétique, la durée maximale.

CE QUE POSTGRESQL VOUS DIT

ERROR: column "film.title" must appear in the GROUP BY clause or be used in an aggregate function

La règle : toute colonne du **SELECT** doit être *soit* citée dans le **GROUP BY**, *soit* passée à une fonction d'agrégation. Ce n'est pas une contrainte arbitraire du langage : c'est la seule façon d'avoir une valeur bien définie par paquet.

Deux réflexes qui évitent la plupart des erreurs

REGROUPEZ SUR LA CLÉ PRIMAIRE

GROUP BY cu.customer_id, cu.last_name

plutôt que **GROUP BY** cu.last_name seul : deux clients homonymes seraient sinon fusionnés dans un seul paquet, et vos comptages seraient faux sans que rien ne le signale.

REGROUPER SUR PLUSIEURS COLONNES

GROUP BY rating, release_year

forme un paquet par **couple** de valeurs. Le résultat compte donc au plus autant de lignes que de couples réellement présents dans la table – pas le produit des deux.

Enfin, rappelez-vous l'ordre d'évaluation (fiche S1) : le **WHERE** s'exécute *avant* le regroupement et filtre des lignes ; le **HAVING** s'exécute *après* et filtre des paquets.

LA RÈGLE DE DÉCISION

Avant d'écrire un **GROUP BY**, formulez la phrase :

« Je veux une ligne **par** ... »

Par catégorie ? **GROUP BY** category_id. Par client ? **GROUP BY** customer_id. Par couple (client, mois) ? les deux colonnes.

Ce qui suit « par » ou « pour chaque » ou équivalent va dans le **GROUP BY** et peut être affiché ; tout le reste doit être agrégé.

VÉRIFIE QUE TU AS COMPRIS

1. Combien de lignes renvoie `SELECT rating FROM film GROUP BY rating` ; sur les 6 films ci-contre ? Et `SELECT DISTINCT rating FROM film` ; ?
2. Cette requête est-elle acceptée ?
`SELECT category_id, COUNT(*) FROM film GROUP BY category_id, title` ; Que renverrait-elle ?
3. Vous voulez « le titre du film le plus long de chaque catégorie ». Peut-on écrire `SELECT category_id, title, MAX(length) ... GROUP BY category_id` ?
4. Que renvoie `SELECT COUNT(*) FROM film` ; sans **GROUP BY** ?

RÉPONSES

1. **3** dans les deux cas : **GROUP BY** sans agrégat et **DISTINCT** donnent ici le même résultat – mais seul le premier permet d'ajouter un `COUNT(*)`. 2. Acceptée, mais inutile : en regroupant aussi par `title`, chaque paquet ne contient qu'un film, donc `COUNT(*)` vaut 1 partout. Plus le **GROUP BY** contient de colonnes, plus les paquets sont petits. 3. **Non** : `title` n'est ni groupé ni agrégé, et `MIN/MAX` sur une autre colonne ne « ramènent » pas la ligne correspondante. Il faut une sous-requête.
4. Une seule ligne : sans **GROUP BY**, toute la table forme un unique paquet.